OP2 Assignment 3 Device driver

May, 2026

Mihnea Bastea

Introduction

In this assignment a simple Linux device driver was tested, modified and extended on a Raspberry Pi system. The starting point was the driver provided by Apriorit, which implements a character device that can be accessed through the `/dev` filesystem.

After compiling and testing the original driver, the device name and buffer contents were modified and additional functionality was added to the driver.

The assignment was developed and tested directly on a Raspberry Pi. Figure 1 shows the system information and development environment.

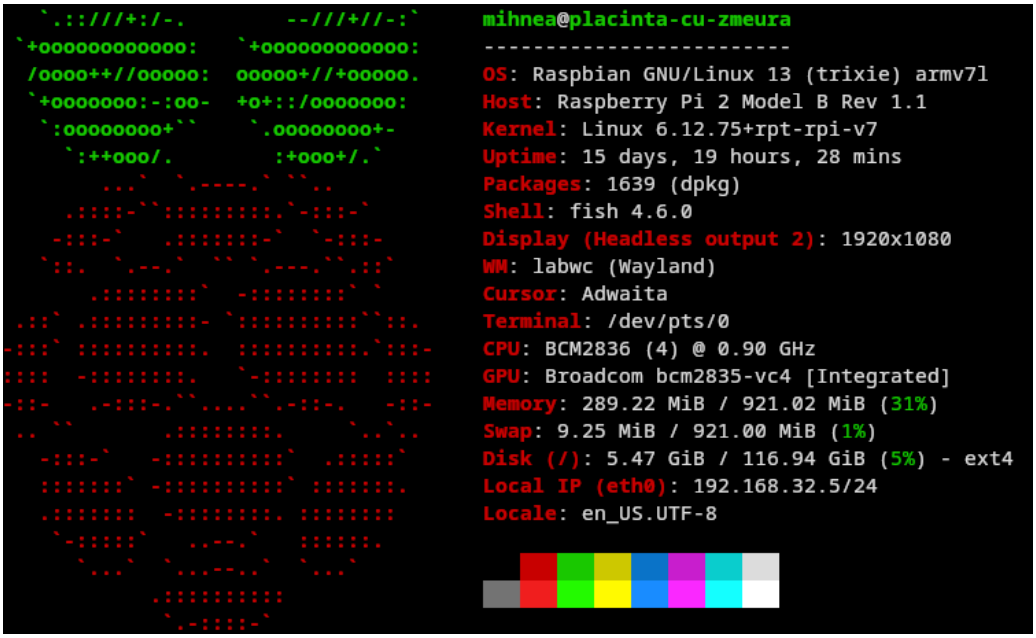

Figure 1: System information obtained using fastfetch.

Part 1a - Original Driver

The original driver from the Apriorit repository was first compiled and tested without modifications. The kernel module was compiled using `make`, which generated the `.ko` kernel object file.

After compilation the module was loaded into the kernel using `insmod`. The kernel log messages were verified using `dmesg` to confirm that the driver initialized correctly and created the device successfully.

```
$ make                                                                    Bash
make -C /lib/modules/6.12.75+rpt-rpi-v7/build M=/home/mihnea/repos/SimpleLinuxDriver
modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v7'
  CC [M]  /home/mihnea/repos/SimpleLinuxDriver/main.o
  CC [M]  /home/mihnea/repos/SimpleLinuxDriver/device_file.o
```

```
  LD [M]  /home/mihnea/repos/SimpleLinuxDriver/simple-module.o
  MODPOST /home/mihnea/repos/SimpleLinuxDriver/Module.symvers
  CC [M]  /home/mihnea/repos/SimpleLinuxDriver/simple-module.mod.o
  CC [M]  /home/mihnea/repos/SimpleLinuxDriver/.module-common.o
  LD [M]  /home/mihnea/repos/SimpleLinuxDriver/simple-module.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v7'
```

Listing 1: Successful compilation of the original driver.

```
$ sudo insmod simple-module.ko
$ dmesg | tail
[1374910.912351] simple_module: loading out-of-tree module taints kernel.
[1374910.912871] Simple-driver: Initialization started
[1374910.912889] Simple-driver: register_device() is called.
[1374910.913429] Simple-driver: Registered character device with major number = 239,
minor number = 0
```

Listing 2: Loading the kernel module and checking kernel log messages.

After loading the module, the device became available through the `/dev` filesystem as `/dev/simple-driver`. The contents of the internal buffer were read using the `cat` command.

```
$ sudo cat /dev/simple-driver
Hello world from kernel mode!
```

Listing 3: Reading data from the original device driver.

Part 1b - Modified Driver

After testing the original driver, the device driver was modified according to the assignment requirements. The device name, class name and kernel module name were changed from `simple-driver` and `simple-module` to `bastea-driver` and `bastea-module`. The contents of the internal buffer were also replaced with a custom buffer message.

The modified values in `device_file.c` are shown below.

```c
static const char device_name[] = "bastea-driver";
static const char class_name[] = "bastea-driver-class";
static const char g_s_Hello_World_string[] = "Hello world from Raspberry Pi device
driver!\n";
```

Listing 4: Modified device name, class name and buffer contents.

The module name in the Makefile was also changed.

```
TARGET_MODULE:=bastea-module
```

Listing 5: Modified kernel module name in the Makefile.

After rebuilding the project, the modified module was loaded into the kernel and verified using `dmesg`.

```
$ make clean
$ make
$ sudo make load
$ dmesg | tail
```

```
[1392475.876173] Simple-driver: register_device() is called.
[1392475.897127] Simple-driver: Registered character device with major number = 239,
minor number = 0

[1392484.895758] bastea-driver: register_device() is called.
[1392484.896324] bastea-driver: Registered character device with major number = 238,
minor number = 0
```

Listing 6: Building and loading the modified driver.

The modified driver was implemented as a separate kernel module while keeping the original driver available. Both modules could be loaded simultaneously and created separate device files inside the `/dev` filesystem.

```bash
$ lsmod | grep module

bastea_module        12288  0
simple_module          12288  0
```

Listing 7: Both kernel modules loaded simultaneously.

The modified device became available through the `/dev` filesystem as `/dev/bastea-driver`. The updated buffer contents were verified using the `cat` command.

```bash
$ sudo cat /dev/bastea-driver

Hello world from Raspberry Pi device driver!
```

Listing 8: Reading the modified device driver.

Test Program

A small test program was created in C to access the modified device driver through the `/dev` filesystem. The program opens the device file using `open()`, reads the contents character-by-character using `read()`, and displays the output using `printf()`.

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char c;

    fd = open("/dev/bastea-driver", O_RDONLY);

    if (fd < 0) {
        perror("Failed to open device");
        return 1;
    }
```

```c
    while (read(fd, &c, 1) > 0) {
        printf("%c", c);
    }

    close(fd);

    return 0;
}
```

Listing 9: Test program used to read from the device driver.

The program was compiled using `gcc` and tested successfully.

```bash
$ gcc read_test.c -o read_test
$ sudo ./read_test

Hello world from Raspberry Pi device driver!
```

Listing 10: Compiling and running the test program.

Part 2 - Driver Extension

For the second part of the assignment the driver was extended with write support. The original driver only supported reading a fixed string from the kernel module. This was changed to a limited writable buffer of 256 bytes.

The fixed constant string was replaced by a writable buffer. A mutex was also added to protect the buffer when it is accessed by the read and write functions.

```c
#define BUFFER_SIZE 256

static char device_buffer[BUFFER_SIZE] = "Hello world from Raspberry Pi device driver!\n";
static DEFINE_MUTEX(buffer_lock);
```

Listing 11: Writable device buffer and mutex.

The `read()` implementation was updated to read from `device_buffer` instead of the original constant string. The current size of the buffer is determined using `strlen()`.

```c
mutex_lock(&buffer_lock);

buffer_size = strlen(device_buffer);

if (*position >= buffer_size) {
    mutex_unlock(&buffer_lock);
    return 0;
}

if (*position + count > buffer_size)
    count = buffer_size - *position;
```

```c
if (copy_to_user(user_buffer, device_buffer + *position, count) != 0) {
    mutex_unlock(&buffer_lock);
    return -EFAULT;
}

*position += count;

mutex_unlock(&buffer_lock);
```

Listing 12: Reading from the writable buffer.

A new `write()` function was added. This function copies data from user space into the kernel buffer using `copy_from_user()`. The number of copied bytes is limited to `BUFFER_SIZE - 1`, so the buffer always has room for a terminating null byte.

```c
static ssize_t device_file_write(struct file *file_ptr, const char __user
*user_buffer, size_t count, loff_t *position){
    size_t bytes_to_copy;

    bytes_to_copy = min(count, (size_t)(BUFFER_SIZE - 1));

    mutex_lock(&buffer_lock);

    memset(device_buffer, 0, BUFFER_SIZE);

    if (copy_from_user(device_buffer, user_buffer, bytes_to_copy) != 0) {
        mutex_unlock(&buffer_lock);
        return -EFAULT;
    }

    device_buffer[bytes_to_copy] = '\0';

    mutex_unlock(&buffer_lock);

    return bytes_to_copy;
}
```

Listing 13: Write implementation.

The write function was then registered in the file operations structure.

```c
static const struct file_operations bastea_driver_fops =
{
    .owner = THIS_MODULE,
    .read = device_file_read,
    .write = device_file_write,
};
```

Listing 14: Registering the write operation.

This makes it possible to write new data to the device file from user space and read the updated contents back afterwards.

To test the extended functionality, a separate C test program was created. The program first reads the initial contents of the device buffer, then writes a new message to the driver and finally reads the updated contents again.

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define DEVICE_PATH "/dev/bastea-driver"

void read_device(void) {
    int fd;
    char c;

    fd = open(DEVICE_PATH, O_RDONLY);

    if (fd < 0) {
        perror("Failed to open device");
        return;
    }

    while (read(fd, &c, 1) > 0) {
        printf("%c", c);
    }

    close(fd);
}

int main(void) {
    int fd;
    const char *message = "Message written from C test program\n";

    printf("Initial buffer contents:\n");
    read_device();

    fd = open(DEVICE_PATH, O_WRONLY);

    if (fd < 0) {
        perror("Failed to open device");
        return 1;
    }

    printf("\nWriting new message to device...\n");
```

```
    write(fd, message, strlen(message));

    close(fd);

    printf("\nUpdated buffer contents:\n");
    read_device();

    return 0;
}
```

Listing 15: Read and write test program.

The test program was compiled using `gcc` and executed successfully.

```
$ gcc read_write_test.c -o read_write_test
$ sudo ./read_write_test

Initial buffer contents:
Hello world from Raspberry Pi device driver!

Writing new message to device...

Updated buffer contents:
Message written from C test program
```

Listing 16: Compiling and running the read/write test program.

Conclusion

During this assignment a Linux character device driver was compiled, modified and extended on a Raspberry Pi system. The original Apriorit driver was first tested without modifications and afterwards adapted with a custom device name and buffer contents.

The driver was then extended with write support using a limited writable buffer. Read and write operations between user space and kernel space were implemented using `copy_to_user()` and `copy_from_user()`. A mutex was added to protect the shared buffer during concurrent access.

The final implementation successfully supported reading and writing through the `/dev` filesystem and was verified using a custom C test program.

Reflection

This assignment helped with understanding how Linux device drivers communicate with user space through the `/dev` filesystem. It also provided practical experience with compiling kernel modules, loading them into the kernel and debugging them using tools such as `dmesg` and `lsmod`.

Extending the original driver with write support made it easier to understand how read and write operations are implemented inside a driver.